

Índice

Trabajo Práctico 2.....	1
Índice	2
Introducción y Objetivo.....	4
Programación Lineal.....	4
1. Análisis del Problema y Objetivo.....	4
2. Supuestos.....	4
3. Variables	5
3. Modelo de Programación Lineal	6
Función Objetivo.....	6
Restricciones	6
Solución y Código :.....	6
Resultado obtenido:.....	7
5. Análisis de la solución.....	7
Redes de Flujo	9
1. Introducción y objetivo	9
2. Supuestos.....	9
3. Diseño	10
a. Modelado como Red de Flujo	10
¿Por qué Edmonds-Karp?.....	11
b. Pseudocódigo	12
Estructuras de datos utilizadas	13
4. Seguimiento	14
Paso 1 — Construcción de la matriz de distancias.....	14
Paso 2 — Construcción del grafo.....	15
Paso 3 — Primera iteración de BFS.....	15
Paso 4 — Actualización de capacidades	16
Paso 5 — Iteraciones siguientes.....	16
Paso 6 — Condición de corte.....	17
Paso 7 — Verificación del resultado	17
5. Complejidad	17
6. Sets de datos	18
7. Tiempos de Ejecución.....	18
Informe de Resultados.....	19

Referencias	31
-------------------	----

Introducción y Objetivo

El presente Trabajo Práctico aborda cuatro problemas de la teoría de algoritmos, cada uno representa un paradigma de diseño distinto: Programación Lineal , Redes de Flujo , Algoritmos de Aproximación y Algoritmos Aleatorios.

En primer lugar, el primer problema consiste en maximizar el beneficio total de una concesión de espacios publicitarios en paradas de colectivos, determinando qué ofertas de clientes aceptar mediante el modelado de Programación Lineal. El segundo problema propone diseñar una red WAN con tolerancia a fallas, asignando conjuntos de antenas de backup sin sobrecargar a ninguna en particular, lo cual requiere adaptar los datos a una red de flujo y resolverlo apoyándose en el algoritmo de Ford-Fulkerson o sus variantes. El tercer problema plantea el desafío de encontrar un subconjunto factible de números enteros cuya suma sea la mayor posible sin superar un límite dado, para lo cual se debe diseñar un algoritmo de aproximación que garantice que su suma total sea al menos la mitad de la suma de cualquier subconjunto óptimo. Por último, el cuarto problema consiste en abordar la versión de optimización del problema de 3-Coloreo en grafos, desarrollando un algoritmo aleatorio de tiempo polinomial que maximice y asegure un número esperado mínimo de aristas satisfechas.

El objetivo principal del trabajo es diseñar, implementar y analizar algoritmos eficientes para cada uno de estos problemas , evaluando empíricamente su complejidad temporal a través de mediciones sobre distintos sets de datos y comparando los tiempos de ejecución obtenidos con las curvas teóricas de ajuste esperadas

Programación Lineal

1. Análisis del Problema y Objetivo

Este problema plantea un modelo de Programación Lineal Entera en el cual se debe maximizar el beneficio de Concesiones Argentina 2000 SRL, una empresa que administra 200 paradas de colectivo y debe decidir a qué clientes asignarles espacios publicitarios durante el próximo mes, respetando ciertas las restricciones de capacidad y compatibilidad entre anunciantes.

2. Supuestos

Para el desarrollo e implementación del modelo de programación lineal se establecieron las siguientes premisas y supuestos con el fin de dejar más en claro el alcance y la base de donde se parte, para ello se tuvo en cuenta lo siguiente:

Sobre el problema:

- Cada parada puede ser asignada a un único cliente. No se permite compartir una parada entre dos anunciantes distintos.
- La asignación es para un período mensual completo: no se contemplan asignaciones parciales en el tiempo.
- El beneficio obtenido por cada cliente es fijo e independiente de factores externos (no hay negociación de precios).

- Las 200 paradas disponibles son homogéneas: no existen diferencias de ubicación, visibilidad o valor entre ellas a los efectos del modelo.

Sobre las variables:

- La decisión de aceptar o rechazar a cada cliente es binaria: no existe una aceptación parcial. Si se acepta al Cliente A, se ocupan exactamente 30 paradas; no se puede ocupar, por ejemplo, 15.
- El Cliente B presenta dos propuestas mutuamente excluyentes (80 paradas por USD 100.000, o 120 paradas por USD 120.000): solo puede seleccionarse una de ellas, o ninguna.

Sobre las restricciones:

- La restricción de incompatibilidad entre A y D se asume como absoluta: no puede haber ninguna parada con publicidad de ambos clientes simultáneamente, independientemente de la cantidad de paradas que ocupe cada uno.
- No existen otras restricciones de incompatibilidad entre clientes más allá de la indicada.

3. Variables

El modelo utiliza variables binarias/indicativas, una por cada opción de cliente disponible. Una variable binaria toma el valor 1 si se acepta esa oferta, y 0 en caso contrario. No se utilizan variables continuas ni enteras, ya que la decisión es de aceptación o rechazo total de cada propuesta.

Variable	Definición	Tipo
x_A	1 si se acepta la oferta del Cliente A (30 paradas, USD 50.000)	Binaria
x_{B1}	1 si se acepta la opción 1 del Cliente B (80 paradas, USD 100.000)	Binaria
x_{B2}	1 si se acepta la opción 2 del Cliente B (120 paradas, USD 120.000)	Binaria
x_C	1 si se acepta la oferta del Cliente C (75 paradas, USD 100.000)	Binaria
x_D	1 si se acepta la oferta del Cliente D (50 paradas, USD 80.000)	Binaria
x_E	1 si se acepta la oferta del Cliente E (2 paradas, USD 5.000)	Binaria
x_F	1 si se acepta la oferta del Cliente F (20 paradas, USD 40.000)	Binaria
x_G	1 si se acepta la oferta del Cliente G (100 paradas, USD 90.000)	Binaria

3. Modelo de Programación Lineal

Función Objetivo

Se busca maximizar el beneficio total obtenido por las concesiones asignadas:

$$\max Z = 50.000 \cdot x_A + 100.000 \cdot x_{B1} + 120.000 \cdot x_{B2} + 100.000 \cdot x_C + 80.000 \cdot x_D \\ + 5.000 \cdot x_E + 40.000 \cdot x_F + 90.000 \cdot x_G$$

Restricciones

- R1) Capacidad total de paradas: La suma de paradas asignadas no puede superar las 200 disponibles.

$$30x_A + 80x_{B1} + 120x_{B2} + 75x_C + 50x_D + 2x_E + 20x_F + 100x_G \leq 200$$

- R2) Exclusividad de opciones del Cliente B: Solo puede seleccionarse una de las dos propuestas del Cliente B, o ninguna.

$$x_{B1} + x_{B2} \leq 1$$

- R3) Incompatibilidad entre Clientes A y D: Por ser competidores directos, no pueden tener publicidad simultánea:

$$x_A + x_D \leq 1$$

Solución y Código :

El modelo fue implementado en lenguaje Python utilizando la biblioteca PuLP para resolución de programas lineales enteros. Los datos del problema se leen desde un archivo llamado "archivo.csv", lo que permite modificar las ofertas sin alterar el código fuente. A continuación, se muestra el código empleado:

```
from pulp import *
import csv
lista_clientes = []
class Cliente:
    def __init__(self, nombre, ganancia, paradas):
        self.nombre = nombre
        self.ganancia = int(ganancia)
        self.paradas = int(paradas)
    def __str__(self):
        return f"{self.nombre} -> {self.ganancia} -> {self.paradas}"
with open('archivo.csv', mode="r") as archivo:
    lector = csv.DictReader(archivo)
    for fila in lector:
        nuevo_cliente = Cliente(
            nombre = fila['nombre'],
```

```
ganancia = fila['ganancia'],
paradas = fila['paradas']
)
lista_clientes.append(nuevo_cliente)
nombres = [c.nombre for c in lista_clientes]
X = LpVariable.dicts("X", nombres,0,cat ="Binary" )
#objetivo de maximizar ganancia
modelo = LpProblem("Propaganda", LpMaximize)
#carga de las retriicciones
modelo += lpSum(i.ganancia * X[i.nombre] for i in lista_clientes),
"max_de_ganancias"
modelo += lpSum(i.paradas * X[i.nombre] for i in lista_clientes) <=
200, "capacidad_paradas"
modelo += X['A'] + X["D"] <= 1, "no simultaneo"
modelo += X['B1'] + X['B2'] <= 1, "B_una_opcion"
modelo.solve()
print("Estado:", LpStatus[modelo.status])
for v in modelo.variables():
    print(v.name, v.varValue)
print("Ganancia:", value(modelo.objective))
```

Resultado obtenido:

```
Estado: Optimal
X_A = 1.0
X_B1 = 1.0
X_B2 = 0.0
X_C = 1.0
X_D = 0.0
X_E = 1.0
X_F = 0.0
X_G = 0.0
Ganancia: 255000.0
```

5. Análisis de la solución

La solución óptima hallada asigna publicidad a los Clientes A, B (opción 1), C y E, obteniendo una ganancia máxima de USD 255.000 con un uso de 187 paradas de las 200 disponibles.

Cliente	Opción elegida	Paradas	Ganancia
A	30 paradas	30	USD 50.000
B1	80 paradas	80	USD 100.000
C	75 paradas	75	USD 100.000

E	2 paradas	2	USD 5.000
Total		187	USD 255.000

Cientes excluidos y justificación:

- Cliente D: excluido por incompatibilidad con A. Como A aporta mejor ratio ganancia/parada (USD 1.667/parada vs USD 1.600/parada de D), el solver prefiere A.
- Cliente B2: descartado en favor de B1 porque, si bien B2 da más ganancia (USD 120.000 vs USD 100.000), ocupa 40 paradas más, lo que impediría incluir a C (que aporta USD 100.000 adicionales). $B1+C = \text{USD } 200.000 > B2 = \text{USD } 120.000$.
- Cientes F y G: con 187 paradas ya ocupadas quedan 13 libres. F necesita 20 y G necesita 100, por lo que ninguno entra.

Redes de Flujo

1. Introducción y objetivo

El presente problema plantea el diseño de una red WAN compuesta por n antenas con tolerancia a fallas. Para garantizar dicha tolerancia, cada antena debe contar con un conjunto de k antenas de backup, seleccionadas entre aquellas que se encuentran a una distancia menor a D . Sin embargo, para evitar que una falla puntual afecte a una porción demasiado grande de la red, se debe imponer una restricción adicional la cual dice que ninguna antena puede aparecer en más de b conjuntos de backup simultáneamente.

El objetivo es dado un conjunto de n antenas con sus coordenadas, y los parámetros k , b y D , determinar una asignación de backups que satisfaga ambas condiciones para todas las antenas, o bien indicar que dicha asignación no es posible.

Para resolver este problema se modela la situación como una red de flujo y se aplica el algoritmo de Edmonds-Karp. La clave de la reducción es que encontrar una asignación válida de backups equivale a encontrar un flujo máximo en una red construida ad hoc: si el flujo máximo alcanza el valor $n \cdot k$, existe solución; de lo contrario, no sería factible o viable.

2. Supuestos

Para el correcto funcionamiento del algoritmo se establecen los siguientes supuestos, condiciones y limitaciones:

Sobre la entrada:

- Los parámetros n , k , b y D son enteros positivos conocidos de antemano.
- Las coordenadas de cada antena son valores numéricos reales en un plano bidimensional. La distancia entre antenas se calcula mediante la distancia euclidiana: $d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$
- Las distancias son simétricas: $d[i, j] = d[j, i]$ para todo par de antenas i, j .
- Una antena no puede ser backup de sí misma: $d[i, j] = 0$, por lo que nunca satisface la condición $d < D$.

Sobre la red de flujo:

- Cada eje $e \in E$ tiene asociada una capacidad c_e que representa el máximo flujo que puede circular por él.
- El flujo en cada eje debe satisfacer: $0 \leq f(e) \leq c_e$ para todo $e=(u,v)$.
- Se cumple la conservación de flujo en todos los nodos intermedios: la suma de flujos entrantes es igual a la suma de flujos salientes.
- Existe un nodo fuente S que genera el flujo necesario y un nodo sumidero T que lo absorbe.

Sobre la solución:

- Si el flujo máximo obtenido es igual a $n \cdot k$, existe una asignación válida de backups y el algoritmo la devuelve.
- Si el flujo máximo es menor a $n \cdot k$, no existe asignación posible que satisfaga todas las restricciones simultáneamente, y el algoritmo lo reporta.
- Si una antena tiene menos de k vecinos a distancia menor a D , la solución no existe para esa instancia, lo cual quedará reflejado en que el flujo máximo no alcanzará $n \cdot k$.

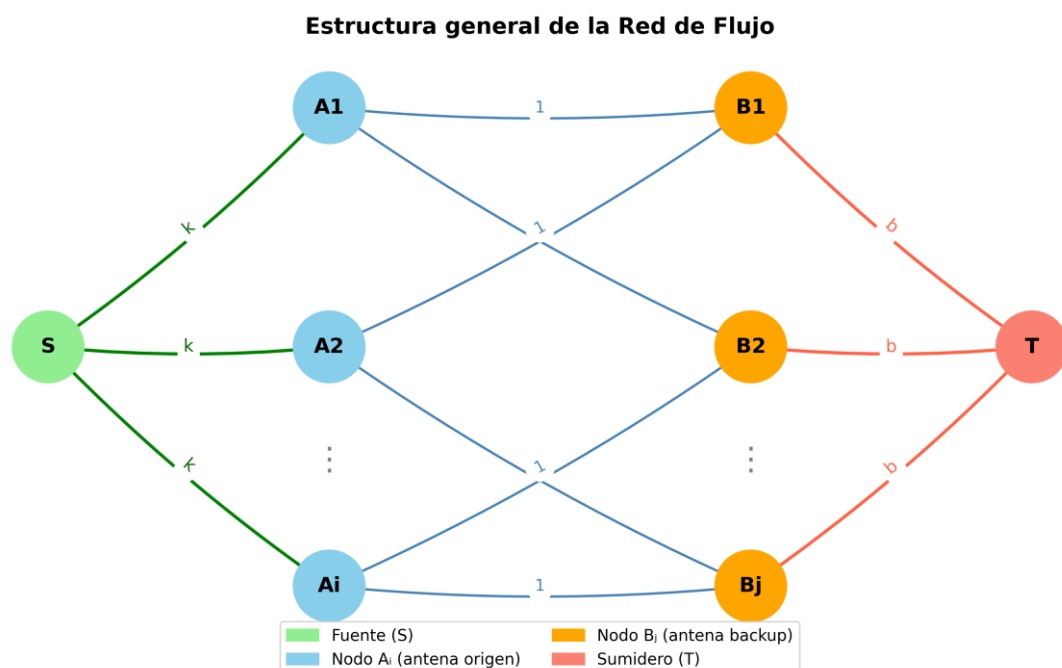
Limitaciones:

- El algoritmo no contempla antenas con coordenadas en más de dos dimensiones.
- No se consideran variaciones dinámicas de la red (altas o bajas de antenas durante la ejecución).
- La entrada se provee en formato JSON con la estructura esperada por el programa; no se realizan validaciones exhaustivas del formato.

3. Diseño

a. Modelado como Red de Flujo

El problema de asignación de backups puede reformularse como un problema de flujo máximo en una red dirigida construida a partir de los datos de entrada. La clave de esta reducción es la siguiente: si existe una asignación válida de backups que respete todas las restricciones, entonces el flujo máximo de la red construida será exactamente $n \cdot k$. Si el flujo máximo es menor, la asignación no existe.



- **Construcción de la red**

Para cada antena i se crean **dos nodos**: A_i y B_i . Esta división en dos nodos por antena es el mecanismo que permite controlar simultáneamente cuántos backups *necesita* cada antena (restricción sobre A_i) y cuántas veces puede *ser* backup de otras (restricción sobre B_i).

Además de los $2n$ nodos de antenas, se agregan dos nodos especiales: una **fente S** y un **sumidero T**.

Las aristas se construyen con las siguientes capacidades, cada una codificando una restricción del problema:

- **$S \rightarrow A_i$ con capacidad k** , para toda antena i . Esta arista controla cuántos backups puede asignarse a la antena i . Al limitar el flujo saliente de A_i a k , se garantiza que cada antena tenga exactamente k backups en la solución final.
- **$A_i \rightarrow B_j$ con capacidad 1**, si y solo si la distancia entre la antena i y la antena j es menor que D (y con $i \neq j$). Esta arista representa la posibilidad de que la antena j sea backup de la antena i . La capacidad 1 garantiza que cada relación de backup se use a lo sumo una vez, evitando asignaciones duplicadas.
- **$B_i \rightarrow T$ con capacidad b** , para toda antena i . Esta arista controla cuántas veces puede la antena i aparecer como backup de otras antenas. Al limitar el flujo entrante a T desde B_i a b , se garantiza que ninguna antena pertenezca a más de b conjuntos de backup simultáneamente.

La red resultante tiene $2n + 2$ nodos y a lo sumo $n^2 + 2n$ aristas.

- **Interpretación de la salida**

Una vez construida la red, se ejecuta el algoritmo de Edmonds-Karp para encontrar el flujo máximo entre S y T . Cada unidad de flujo que circula por un camino $S \rightarrow A_i \rightarrow B_j \rightarrow T$ representa la decisión de asignar a la antena j como backup de la antena i .

Al finalizar la ejecución, se compara el flujo máximo obtenido con el valor $n \cdot k$:

Si **flujo máximo** = $n \cdot k$, cada antena recibió exactamente k backups válidos respetando el límite b . La solución existe y se puede leer directamente del flujo: para cada arista $A_i \rightarrow B_j$ que tenga flujo 1 (es decir, cuya capacidad residual sea 0), la antena j es backup de la antena i .

Si **flujo máximo** < $n \cdot k$, no fue posible asignar k backups válidos a todas las antenas respetando el límite b . El algoritmo reporta que no existe solución posible para los parámetros dados.

¿Por qué Edmonds-Karp?

Edmonds-Karp es la variante de Ford-Fulkerson que utiliza BFS para encontrar el camino de aumento más corto en cada iteración. Esto garantiza que el algoritmo termina en tiempo polinomial, independientemente de las capacidades de la red. Su complejidad es $O(V \cdot E^2)$, lo que combinado con la estructura de la red construida produce la complejidad total del algoritmo, analizada en detalle en la sección correspondiente.

b. Pseudocódigo

A continuación, se muestra el pseudocódigo que se realizó para este problema:

```
Definir antenas como Lista de Antena
Definir n como Entero de antenas
Definir matrix_distancia como Matrix de n * n
Definir G como GrafoDirigido
Definir flujoMaximo como Entero
Definir antenas_usadas como Diccionario

Funcion Principal()
  Leer JSON
  A <- data["antenas"]
  D <- data["D"]
  b <- data["b"]
  k <- data["k"]
  para cada antena en A hacer:
    Agregar (antena como objeto Antena) a antenas
  matrix_distancia <- armarMatrixDistancia(antenas)
  grafo <- grafoDirigido(matrix_distancia, antenas, D, k, b)
  flujoMaximo <- 0
  padres <- bfs(grafo, "S", "T")
  Mientras padres != Falso hacer:
    camino <- reconstruirCamino(padres, "S", "T")
    cuello <- cuelloBotella(camino, grafo)
    actualizarCapacidades(camino, grafo, cuello, antenas_usadas)
    flujoMaximo <- flujoMaximo + cuello
    padres <- bfs(grafo, "S", "T")
  Si flujoMaximo = n * k Entonces
    retornar antenas_usadas
  Sino
    retornar "No existe solución"
FinFuncion

Funcion cuelloBotella(camino, grafo)
  Definir cuello como Lista
  para cada i <- 0 hasta longitud(camino)-2
    u <- camino[i]
    v <- camino[i+1]
    Agregar (capacidad(u,v)) a cuello
  Retornar menor elemento de cuello
FinFuncion

Funcion actualizarCapacidades(camino, grafo, flujo, antenas_usadas)
  para cada e=(u,v) ∈ camino Hacer:
    capacidad(u,v) <- capacidad(u,v) - flujo
    Si no existe e=(v,u) Entonces:
      Agregar ((v,u) con capacidad de flujo) a grafo
    Sino:
      capacidad(v,u) <- capacidad(v,u) + flujo
    Si u != S y v != T Entonces:
      Agregar v como backup de u en antenas_usadas
  FinPara
FinFuncion

Funcion armarMatrixDistancia(antenas)
```

```
Definir n como cantidad de antenas
Definir matrix como Matriz de tamaño n * n
para cada i en antenas Hacer:
    Definir fila como Lista
    para cada j en antenas Hacer:
        Agregar (i.distancia(j)) a fila
    Agregar (fila) a matrix
Retornar matrix
FinFuncion

Funcion grafoDirigido(matrix_distancia, antenas, D, k, b)
Definir G como GrafoDirigido
Para cada antena i Hacer:
    Agregar arista(S, Ai, capacidad = k) a G
    Agregar arista(Bi, T, capacidad = b) a G
Para cada i en cantidad de antenas Hacer:
    Para cada j en cantidad de antenas Hacer:
        Si i != j y matrix_distancia[i][j] < D Entonces:
            Agregar arista(Ai, Bj, capacidad = 1) a G
Retornar G
FinFuncion

Funcion bfs(grafo, s, t)
Definir cola como Cola
Definir visitados como Conjunto
Definir padres como Diccionario
cola <- [s]
Agregar (s) a visitados
Mientras cola no este vacia Hacer:
    nodo <- desencolar(cola)
    Si nodo == t Entonces:
        Retornar padres
    Para cada vecino en grafo[nodo] Hacer:
        Si vecino no está en visitados y capacidad(nodo, vecino) > 0 Hacer:
            Agregar (vecino) a visitados
            padres[vecino] <- nodo
            encolar(cola, vecino)
Retornar Falso
FinFuncion

Funcion reconstruirCamino(padres, s, t)
Definir camino como Lista
actual <- t
Mientras actual != s Hacer:
    Agregar (actual) a camino
    actual <- padres[actual]
Agregar (s) a camino
Revertir camino
Retornar camino
FinFuncion
```

Estructuras de datos utilizadas

- **Clase Antena:** representa cada antena de la red con sus atributos de identificador y coordenadas (x, y). Provee además el método distancia() para calcular la distancia euclidiana entre dos antenas.

- **Lista de antenas:** almacena los objetos Antena cargados desde el archivo JSON. Se recorre secuencialmente para construir la matriz de distancias y el grafo.
- **Matriz de distancias:** matriz cuadrada de $n \times n$ donde la posición $[i][j]$ contiene la distancia euclidiana entre la antena i y la antena j . Se utiliza para determinar qué aristas $A_i \rightarrow B_j$ se agregan al grafo.
- **Grafo dirigido (GrafoDirigido):** representado internamente como un diccionario de diccionarios, donde $G[u][v]['capacity']$ indica la capacidad residual de la arista $u \rightarrow v$. Esta estructura permite acceder y modificar capacidades en $O(1)$ y recorrer vecinos eficientemente en el BFS.
- **Cola (deque):** utilizada en BFS para procesar nodos en orden de llegada, garantizando que se encuentre siempre el camino más corto en cantidad de aristas, lo cual es el requisito central de Edmonds-Karp.
- **Conjunto de visitados:** utilizado en BFS para evitar visitar el mismo nodo más de una vez por iteración.
- **Diccionario de padres:** utilizado en BFS para registrar el nodo predecesor de cada nodo visitado, permitiendo reconstruir el camino de aumento una vez alcanzado el sumidero T .
- **Diccionario antenas_usadas:** almacena la asignación final de backups. La clave es el nodo A_i y el valor es la lista de nodos B_j asignados como backup de esa antena. Se construye durante *actualizarCapacidades* cada vez que una arista $A_i \rightarrow B_j$ es utilizada por un camino de aumento.

4. Seguimiento

Para ilustrar el funcionamiento del algoritmo se utiliza el siguiente conjunto de datos reducido:

```
{
  "D": 5,
  "k": 2,
  "b": 3,
  "antenas": [
    {"id": 1, "x": 0, "y": 0},
    {"id": 2, "x": 1, "y": 1},
    {"id": 3, "x": 2, "y": 0}
  ]
}
```

Paso 1 — Construcción de la matriz de distancias

Se calcula la distancia euclidiana entre cada par de antenas:

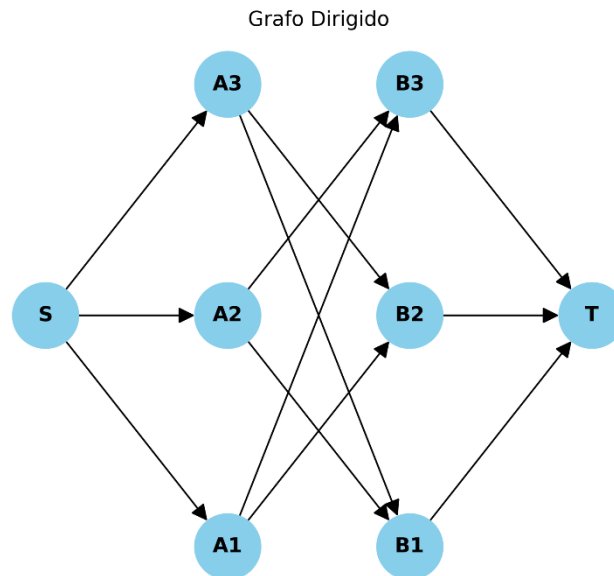
	a1	a2	a3
a1	0	1.41	2
a2	1.41	0	1.41
a3	2	1.41	0

Como todas las distancias entre antenas distintas son menores a $D=5$, todas las combinaciones posibles de backup son válidas.

Paso 2 — Construcción del grafo

Se construye el grafo dirigido con $2n+2 = 8$ nodos: S, A1, A2, A3, B1, B2, B3 y T. Las aristas se agregan con las siguientes capacidades:

- $S \rightarrow A1, S \rightarrow A2, S \rightarrow A3$ con capacidad $k=2$
- $A1 \rightarrow B2, A1 \rightarrow B3, A2 \rightarrow B1, A2 \rightarrow B3, A3 \rightarrow B1, A3 \rightarrow B2$ con capacidad 1 (todas las distancias son menores a $D=5$)
- $B1 \rightarrow T, B2 \rightarrow T, B3 \rightarrow T$ con capacidad $b=3$



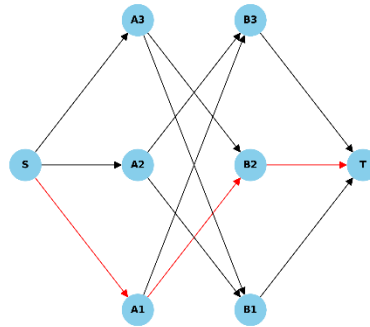
Paso 3 — Primera iteración de BFS

Se ejecuta BFS desde S hasta T y se encuentra el primer camino de aumento: $S \rightarrow A1 \rightarrow B2 \rightarrow T$. El cuello de botella es 1, limitado por la arista $A1 \rightarrow B2$. Se actualizan las capacidades restando 1 en el sentido del flujo y creando la arista inversa correspondiente.

Camino de aumento con: $k = 2, b = 3$

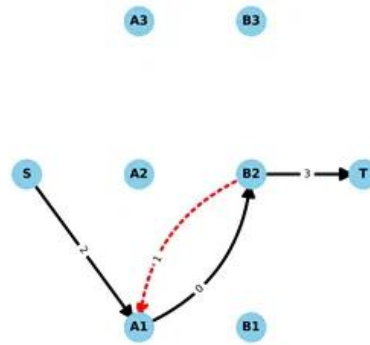


El camino encontrado sobre el grafo completo se ve de la siguiente manera



Paso 4 — Actualización de capacidades

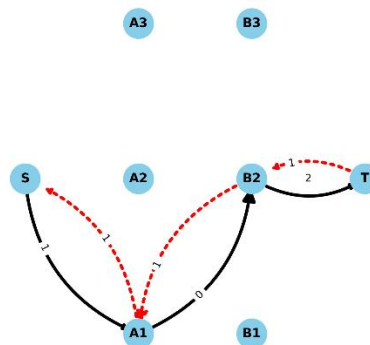
Tras la primera iteración el flujo acumulado es 1. La arista $A1 \rightarrow B2$ queda con capacidad residual 0, indicando que fue utilizada. Se crea la arista inversa $B2 \rightarrow A1$ con capacidad 1, que permite deshacer esta asignación en iteraciones futuras si el algoritmo lo requiere. En el diagrama se puede observar el estado de la red residual tras esta primera iteración: la arista $S \rightarrow A1$ muestra capacidad 2 (capacidad original) y $B2 \rightarrow T$ muestra capacidad 3 (capacidad original), mientras que la arista inversa $B2 \rightarrow A1$ con capacidad 1 refleja el flujo enviado.



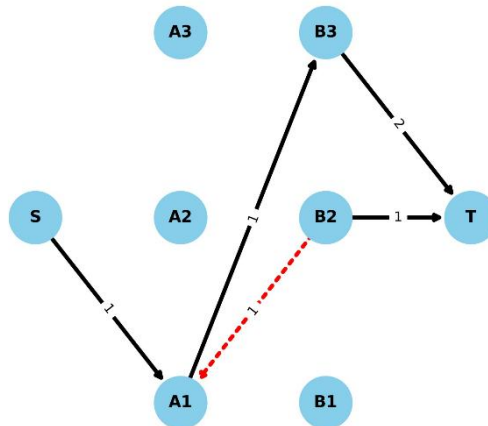
Paso 5 — Iteraciones siguientes

El algoritmo repite el proceso encontrando nuevos caminos de aumento hasta que no existan más. En cada iteración el flujo se incrementa en 1 unidad.

Segunda iteración: BFS encuentra el camino $S \rightarrow A1 \rightarrow B3 \rightarrow T$ con cuello de botella 1. El flujo acumulado pasa a 2. Se actualizan las capacidades residuales: $A1 \rightarrow B3$ queda con capacidad 0 y se crea la arista inversa $B3 \rightarrow A1$ con capacidad 1. La arista $B3 \rightarrow T$ pasa de 3 a 2.



Tras esta iteración, A1 ya tiene sus 2 backups asignados (B2 y B3), por lo que $S \rightarrow A1$ queda con capacidad 0 y no puede recibir más flujo. El diagrama muestra el estado de la red tras las primeras dos iteraciones. Las iteraciones siguientes continúan por A2 y A3, siguiendo el mismo procedimiento, hasta alcanzar el flujo total $n \cdot k = 6$. Por claridad, estos pasos intermedios no se ilustran individualmente ya que siguen el mismo patrón.



Paso 6 — Condición de corte

BFS ya no encuentra caminos de aumento entre S y T. El algoritmo finaliza con flujo máximo = 6.

Paso 7 — Verificación del resultado

El flujo máximo obtenido es $n \cdot k = 3 \cdot 2 = 6$. Como el flujo alcanzó exactamente ese valor, existe una asignación válida de backups. La asignación resultante se lee del diccionario *antenas_usadas*:

- A1 tiene como backups: B2, B3
- A2 tiene como backups: B1, B3
- A3 tiene como backups: B1, B2

Se puede verificar que la solución respeta todas las restricciones del problema: cada antena tiene exactamente $k=2$ backups asignados, todos a distancia menor a $D=5$, y ninguna antena aparece en más de $b=3$ conjuntos de backup simultáneamente (B1 aparece en 2, B2 en 2, B3 en 2).

5. Complejidad

La complejidad temporal total del algoritmo resulta de sumar los costos de sus tres etapas principales.

La construcción de la matriz de distancias requiere calcular la distancia euclidiana entre cada par de antenas, lo que implica $O(n^2)$ operaciones. La construcción del grafo dirigido recorre la matriz para agregar las aristas, con el mismo costo $O(n^2)$.

La red de flujo construida contiene $2n+2$ vértices: dos nodos por antena (A_i y B_i), más la fuente S y el sumidero T, resultando en $V = O(n)$. En el peor caso, cada antena puede conectarse con todas las demás, generando $n(n-1)$ aristas entre los conjuntos A y B. Sumando las n aristas desde S y las n aristas hacia T, la cantidad total de aristas es $E = O(n^2)$.

Posteriormente se ejecuta Edmonds-Karp, cuya complejidad es $O(V \cdot E^2)$. Reemplazando $V = O(n)$ y $E = O(n^2)$:

$$O(V \cdot E^2) = O(n \cdot (n^2)^2) = O(n^5)$$

La complejidad temporal total queda entonces:

$$T(n) = O(n^2) + O(n^2) + O(n^5) = O(n^5)$$

Siendo $O(n^5)$ el término dominante.

6. Sets de datos

Los sets de datos fueron generados programáticamente con coordenadas aleatorias mediante semilla fija “*random.seed(42)*”, lo que garantiza la reproducibilidad de los resultados en cualquier entorno de ejecución. Las coordenadas de cada antena se generan en el rango $[0, 100]$. El programa recibe como argumento el archivo JSON correspondiente y lo procesa ejecutando `python3 antena.py <archivo.json>`.

Se diseñaron 6 conjuntos de datos de distintos tamaños, variando también los parámetros k , b y D para explorar distintos comportamientos del algoritmo. En particular, el set de 500 antenas utiliza $k=1$ y $b=5$ en lugar de $k=2$ y $b=3$, con el objetivo de observar cómo la reducción en la cantidad de backups requeridos afecta el tiempo de ejecución.

Archivo	n	D	k	b	Flujo obtenido	Flujo necesario	Resultado
antenas_100.json	100	20	2	3	200	200	Solución válida
antenas_200.json	200	20	2	3	400	400	Solución válida
antenas_300.json	300	20	2	3	600	600	Solución válida
antenas_400.json	400	20	2	3	800	800	Solución válida
antenas_500.json	500	20	1	5	500	500	Solución válida
antenas_600.json	600	20	2	3	1200	1200	Solución válida

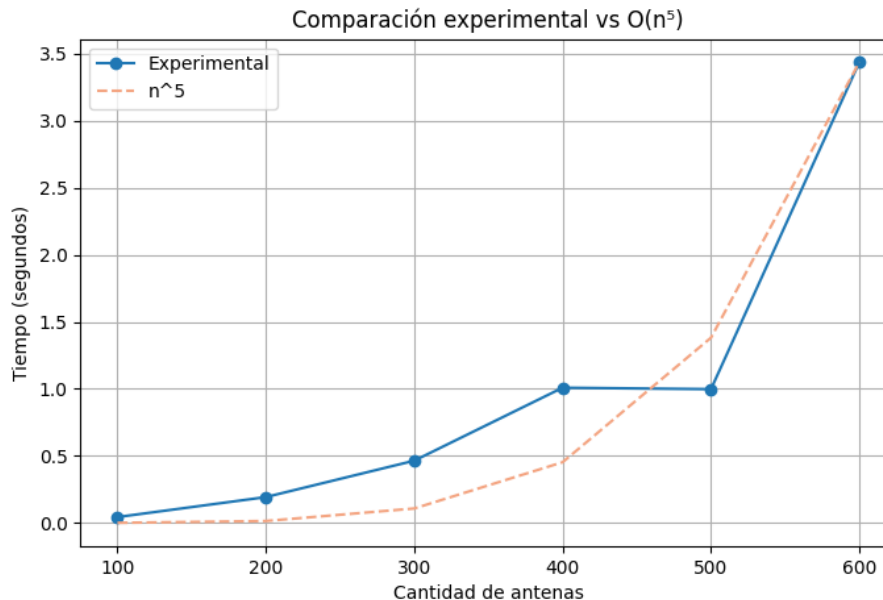
En todos los casos el flujo máximo obtenido coincidió con el flujo necesario $n \cdot k$, confirmando que existe una asignación válida de backups para cada instancia. Los archivos JSON de cada set y sus resultados completos se incluyen en la entrega.

7. Tiempos de Ejecución

Se midieron los tiempos de ejecución para conjuntos de datos de distintos tamaños, generando las antenas con coordenadas aleatorias mediante semilla fija para garantizar la reproducibilidad de los resultados.

Antenas (n)	Flujo Necesario	Tiempo (segundos)
100	200	0.0434
200	400	0.1924
300	600	0.4640
400	800	1.0083
500	500	0.9975
600	1200	3.4372

El gráfico comparativo muestra la curva experimental junto a la curva teórica $O(n^5)$ escalada:



Informe de Resultados

Los tiempos medidos evidencian un crecimiento acelerado que acompaña la forma de la curva teórica $O(n^5)$. Sin embargo, el comportamiento más interesante y revelador del algoritmo se observa al comparar las instancias de $n=400$ y $n=500$.

A pesar de que el grafo de $n=500$ posee una mayor cantidad de vértices y de aristas potenciales, su tiempo de ejecución experimenta una disminución respecto al de $n=400$ (0.997s frente a 1.008s). Esto consolida la hipótesis planteada en el diseño: la complejidad práctica de Edmonds-Karp es sumamente sensible al valor del flujo máximo a alcanzar. Para el set de 500 antenas, al reducir el requerimiento de backups a $k=1$ (lo que implica un flujo total a encontrar de 500) y relajar el límite de uso a $b=5$, el algoritmo necesita descubrir considerablemente menos caminos de aumento en la red residual comparado con el caso de $n=400$ (donde $k=2$, exigiendo un flujo total de 800). Esto demuestra empíricamente que $O(n^5)$ representa una cota superior para el peor caso, pero las instancias con menor exigencia de flujo se resuelven de manera mucho más eficiente, interrumpiendo la tendencia de crecimiento polinomial puro.

En conclusión, la reducción del problema de asignación de backups tolerantes a fallos mediante una red de flujo resultó ser una estrategia algorítmica robusta y correcta. La estructura bipartita del grafo (A_i y B_j) garantizó el cumplimiento estricto de las restricciones operativas de la red WAN: un máximo de b usos por antena y exactamente k backups a una distancia menor a D . Las pruebas empíricas no solo confirmaron la validez de las soluciones obtenidas (alcanzando el flujo máximo $n \cdot k$ en todos los casos factibles), sino que permitieron evidenciar la naturaleza dependiente del flujo que caracteriza a Edmonds-Karp. Esto nos permitió validar la precisión de la cota asintótica superior de $O(n^5)$ y comprender cómo variaciones paramétricas favorables optimizan drásticamente los tiempos de ejecución en la práctica.

Algoritmos de Aproximación

Introducción y Objetivo

El presente problema aborda el diseño y análisis de un algoritmo de aproximación para una variante del problema de optimización conocido como la suma de subconjuntos (*Subset Sum*). El objetivo principal consiste en encontrar un subconjunto factible S a partir de un conjunto de enteros positivos A , tal que la suma de sus elementos sea la mayor posible sin exceder un límite o capacidad dada B .

Inicialmente, se parte del análisis de un algoritmo ávido (*greedy*) rudimentario que procesa e incorpora secuencialmente los elementos mientras la restricción de capacidad lo permita. Al evidenciarse que este enfoque básico no garantiza una solución de calidad acotada, el trabajo se centra en el diseño de un algoritmo de aproximación superador. El objetivo fundamental de este nuevo diseño es garantizar matemáticamente que la suma total obtenida sea siempre, en el peor de los casos, al menos la mitad de la suma de cualquier subconjunto óptimo ($OPT/2$). Adicionalmente, se analizará teórica y empíricamente la complejidad temporal de la solución propuesta para validar su eficiencia.

1. Análisis del Algoritmo Original y Caso de Falla

El problema plantea un conjunto de enteros positivos $A = \{a_1, a_2 \dots a_n\}$ y una capacidad B . Se busca un subconjunto factible $S \subseteq A$ que maximice la suma de sus elementos sin superar B .

El algoritmo original propuesto para resolver esto evalúa y agrega los elementos secuencialmente:

```
subconjunto_factible(A, B)
  S={ }
  T=0
  Para i=1,2,...,n
    Si T + A[i] <= B, entonces:
      S = S ∪ A[i]
      T = T + A[i]
  fin si
fin para
devolver S
```

Lo que se pide es encontrar un caso empírico que demuestre que la solución provista por el algoritmo no es óptima ni garantiza una cota de $1/2$. Considerando que el algoritmo depende estrictamente del orden de los elementos del conjunto (pues los va incorporando a la solución a medida que los procesa), una forma de encontrar las instancias pedidas es que el conjunto A esté formado por solo dos elementos y elegir un límite B que produzca dos subconjuntos factibles compuestos por un solo elemento.

Sea $A = \{x_1, x_2\}$. Para que el algoritmo seleccione x_1 y luego deba rechazar x_2 , definimos la capacidad como el elemento mayor: $B = x_1$.

Para que se cumpla la falla de la consigna, la suma del elemento elegido por el algoritmo (x_1) debe ser menor a la mitad del óptimo (x_2):

$$x_1 < \frac{1}{2} x_2 \Rightarrow x_2 > 2x_1$$

Elegir un x_1 nos dará una pauta de cómo debe ser x_2 . Por ejemplo, si fijamos $x_1 = 1$, necesitamos un $x_2 > 2$.

Instancia propuesta:

$$A = \{1, 6\}$$
$$B = 6$$

Los subconjuntos factibles son $S_1 = \{1\}$ Y $S_2 = \{1\}$. El algoritmo devolverá S_1 por tratarse del primer elemento que se evalúa y cabe en la capacidad B, cuando en realidad S_2 es la solución óptima.

Esta instancia cumple estrictamente con la consigna, ya que:

$$\sum_{x \in S_1} x < \frac{1}{2} \sum_{x \in S_2} x$$

Pues $1 < \frac{1}{2} (6)$, es decir, $1 < 3$.

2. Diseño del Algoritmo de Aproximación

Para solucionar la falla del algoritmo original (donde un elemento pequeño puede bloquear la entrada de uno mucho más grande y arruinar el resultado), proponemos un nuevo algoritmo con una mejora sencilla.

La idea principal es ejecutar el algoritmo Greedy original, pero al mismo tiempo identificar cuál es el elemento individual más grande que entra en nuestra capacidad límite. Al final, se compara ambas opciones y nos quedamos con la que suma más.

El algoritmo propuesto se compone de los siguientes pasos:

1. **Ejecutar el recorrido Greedy:** Hacemos el mismo proceso secuencial del algoritmo original sobre el conjunto A para obtener un subconjunto factible S_g , cuya suma total llamaremos G.
2. **Buscar el mayor elemento individual:** Identificamos cuál es el valor máximo individual del conjunto, al que llamaremos M, que cumpla con la restricción del problema (es decir, que no se pase de la capacidad: $M \geq B$).
3. **Devolver la mejor solución:** Comparamos las dos alternativas. Si lo que juntó el algoritmo Greedy es mayor o igual al elemento individual más grande ($G \geq M$), el algoritmo devuelve el subconjunto S_g . En caso contrario, descartamos la selección Greedy y devolvemos un subconjunto formado únicamente por el elemento M.

Este paso extra de guardar el "plan B" (el mayor elemento por sí solo) es lo que nos va a permitir garantizar que nunca nos quedemos con un resultado pobre y que nuestra suma siempre sea, en el peor de los casos, al menos la mitad del óptimo.

3. Supuestos

Para el diseño y funcionamiento de este algoritmo, partimos de las siguientes premisas y condiciones:

- **Naturaleza de los datos:** Todos los elementos del conjunto A y la capacidad límite B son enteros positivos. Esto nos asegura que al agregar un elemento, la suma siempre se incrementa, haciendo predecible el comportamiento secuencial.

- **Condición de factibilidad:** Un subconjunto se considera factible únicamente si la suma total de sus elementos es menor o igual a B .
- **Descarte directo:** Cualquier elemento del conjunto A cuyo valor individual sea estrictamente mayor que B es ignorado por completo, ya que por sí solo viola la restricción de capacidad y jamás podría formar parte de una solución factible.
- **Procesamiento secuencial:** El algoritmo recibe y evalúa los elementos en el orden exacto en el que vienen dados, sin necesidad de aplicar algoritmos de ordenamiento previo (lo cual nos permite mantener una complejidad temporal baja).
- **Meta:** El objetivo de optimización es maximizar la suma total del subconjunto seleccionado.

4. Diseño

a. Cumplimiento de la garantía solicitada

Llamemos OPT a la suma total del subconjunto óptimo (sabemos que, por definición, $OPT \leq B$).

Nuestro algoritmo siempre va a devolver el máximo entre dos valores: $\max(G, M)$, donde G es la suma obtenida por el recorrido Greedy, y M es el mayor elemento individual factible.

Se quiere demostrar matemáticamente que la solución siempre cumple la garantía:

$$\max(G, M) \geq \frac{OPT}{2}$$

La forma más clara de probar esto es por el método de reducción al absurdo (contradicción).

1. Suponemos por un momento que nuestro algoritmo falla y no cumple la garantía. Es decir, supongamos que el resultado que devolvemos es menor a la mitad del óptimo:

$$G < \frac{OPT}{2} \text{ y también } M < \frac{OPT}{2}$$

2. Si $M < \frac{OPT}{2}$ esto significa que absolutamente todos los elementos factibles del conjunto miden menos que la mitad del óptimo (ya que M es el más grande de todos ellos).
3. Ahora, pasando a lo que realizó el algoritmo Greedy. Si terminó con una suma $G < \frac{OPT}{2}$, significa que dejó libre más de la mitad de la capacidad óptima.
4. Como el algoritmo Greedy no logró alcanzar la suma óptima, en algún momento de su recorrido tuvo que haber rechazado algún elemento (llamémoslo x) que sí forma parte de la solución óptima.

5. La única razón por la que Greedy rechaza un elemento es porque, al intentar sumarlo, se pasaba de la capacidad B . Es decir, en ese instante:

$$\text{Suma acumulada} + x > B$$

6. Sin embargo, por nuestra suposición inicial sabemos que la suma acumulada nunca superó G (que es $< \frac{OPT}{2}$) y que el elemento x no puede ser mayor que M (que también es $< \frac{OPT}{2}$). Si sumamos ambos peores casos, obtenemos:

$$\text{Suma acumulada} + x < \frac{OPT}{2} + \frac{OPT}{2} = OPT$$

7. Como el óptimo siempre es menor o igual a la capacidad ($OPT \leq B$), llegamos a la conclusión de que:

$$\text{Suma acumulada} + x > B$$

8. Pero justamente aquí se encuentra la contradicción. Es imposible que la suma acumulada más el elemento x sea menor a B y que, al mismo tiempo, el algoritmo de Greedy lo haya rechazado por pasarse de B . El algoritmo de Greedy jamás lo habría dejado afuera si había espacio.

Por lo tanto, nuestra suposición inicial es falsa. Queda demostrado que es imposible que tanto G como M se queden por debajo de la mitad del óptimo. Necesariamente, al menos uno de los dos va a ser $\geq \frac{OPT}{2}$, y como nosotros devolvemos el máximo entre ambos, la garantía siempre se cumple.

b. Pseudocódigo

El algoritmo recorre el conjunto de entrada una única vez, manteniendo actualizadas tanto la solución Greedy como el mayor elemento individual factible en cada paso:

```
Algoritmo aproximacion_subconjunto(A, B):  
  Sg = {}  
  G = 0  
  M = 0  
  mejor_elemento = nulo  
  
  Para i = 1 hasta n hacer:  
    // 1. Buscamos el mayor elemento individual factible  
    Si A[i] <= B y A[i] > M entonces:  
      M = A[i]  
      mejor_elemento = A[i]  
  
    // 2. Lógica Greedy original  
    Si G + A[i] <= B entonces:  
      Sg = Sg ∪ {A[i]}  
      G = G + A[i]
```

```
// 3. Devolvemos la mejor de las dos opciones
Si G >= M entonces:
    devolver Sg
Si no:
    devolver {mejor_elemento}
```

c. Estructuras de datos utilizadas

Para mantener la eficiencia del algoritmo y garantizar un tiempo de ejecución lineal, se optó por estructuras simples que permiten accesos y modificaciones en tiempo constante $O(1)$:

- **Arreglo o Lista (A):** Almacena los elementos de entrada. Permite la iteración secuencial elemento por elemento.
- **Conjunto o Lista (Sg):** Estructura dinámica que almacena los elementos que van siendo seleccionados y aprobados por la heurística Greedy.
- **Variable numérica (G):** Funciona como un acumulador en tiempo real de la suma de los elementos dentro de Sg, evitando tener que recalcular la suma total del conjunto en cada iteración.
- **Variable numérica (M):** Almacena y actualiza el valor del mayor elemento individual factible encontrado hasta el momento.
- **Variable auxiliar (mejor_elemento):** Guarda la referencia o el valor exacto del elemento asociado a M para poder construir y devolver el subconjunto unitario en caso de que esta opción resulte ganadora al final de la ejecución.

5. Seguimiento

Para ilustrar el comportamiento del algoritmo, realizaremos un seguimiento paso a paso con un conjunto de datos reducido. Este ejemplo está pensado específicamente para mostrar cómo nuestra mejora logra corregir la falla de la heurística Greedy pura.

Instancia de prueba:

- Conjunto de entrada: $A = \{1, 100, 20, 30\}$
- Capacidad límite: $B = 100$

Paso 1: Ejecución del recorrido principal

El algoritmo comienza a iterar sobre el arreglo A. En cada paso, evalúa si el elemento entra en la capacidad disponible (lógica Greedy) y, en paralelo, rastrea cuál es el mayor elemento individual que no se pasa de B.

Elemento evaluado	Suma acumulada previa	Lógica Greedy: ¿Se agrega?	Rastreo del Mayor Elemento (M)
1	0	Sí ($0 + 1 \leq 100$)	Se actualiza: $M = 1$
100	1	No ($1 + 100 > 100$)	Se actualiza: $M = 100$
20	1	Sí ($0 + 20 \leq 100$)	Se mantiene: $M = 100$
30	21	Sí ($21 + 30 \leq 100$)	Se mantiene: $M = 100$

Paso 2: Evaluación de resultados parciales

Al finalizar el ciclo, el algoritmo tiene dos candidatos en la mano:

- **Resultado del recorrido Greedy:** Logró armar el subconjunto $Sg = \{1, 20, 30\}$, alcanzando una suma total de $G = 51$.
- **Mayor elemento individual factible:** Encontró que el mejor número por sí solo es el 100, por lo que $M = 100$.

Paso 3: Selección final

El algoritmo ejecuta la condición final preguntándose cuál de las dos estrategias recolectó más valor. Como la suma Greedy no es mayor o igual al elemento individual el algoritmo descarta el subconjunto Greedy y devuelve como solución el subconjunto unitario formado por el mayor elemento.

- Resultado final devuelto: $S = \{100\}$
- Suma total obtenida: $ALG = 100$

Conclusión del ejemplo:

Como se puede observar, si hubiéramos dependido exclusivamente del algoritmo Greedy original, habríamos obtenido una suma de apenas 51, desperdiciando casi la mitad de la capacidad por culpa de haber aceptado el número 1 en el primer paso. Gracias a la validación en paralelo del mayor elemento individual, nuestro algoritmo logra rescatar el 100 y alcanzar la solución óptima perfecta, cumpliendo con la garantía.

6. Complejidad

Complejidad Temporal

El análisis del tiempo de ejecución se deduce de forma directa a partir de la estructura de control del pseudocódigo:

1. **Inicialización:** La declaración e inicialización de las estructuras y variables base (Sg, G, M y mejor_elemento) requiere un tiempo de ejecución constante, es decir, $O(1)$.
2. **Ciclo principal:** El algoritmo emplea una única estructura iterativa que recorre el arreglo de entrada A exactamente n veces, siendo n la cantidad total de elementos.
3. **Operaciones elementales:** Dentro del ciclo, las sentencias se limitan a evaluaciones de condiciones lógicas (comparaciones contra B y M), cálculos aritméticos básicos (acumulación en G) y asignaciones de variables. Todas estas instrucciones son operaciones elementales procesadas en tiempo constante $O(1)$.
4. **Costo de inserción:** La eventual adición de un elemento a la solución parcial Sg (implementado convencionalmente como un arreglo dinámico o lista) posee un costo de inserción amortizado de $O(1)$.

Sumando los aportes, el tiempo total está estrictamente acotado por $O(1) + n.O(1)$. En consecuencia, el término dominante es el recorrido lineal del arreglo, lo que arroja una complejidad temporal final de $O(n)$.

7. Sets de datos

Para evaluar el rendimiento del algoritmo, se generaron conjuntos de datos aleatorios de distintos tamaños (n). Con el objetivo de garantizar la reproducibilidad exacta de los experimentos en cualquier entorno de prueba, se fijó una semilla de generación pseudoaleatoria (*random.seed(42)*).

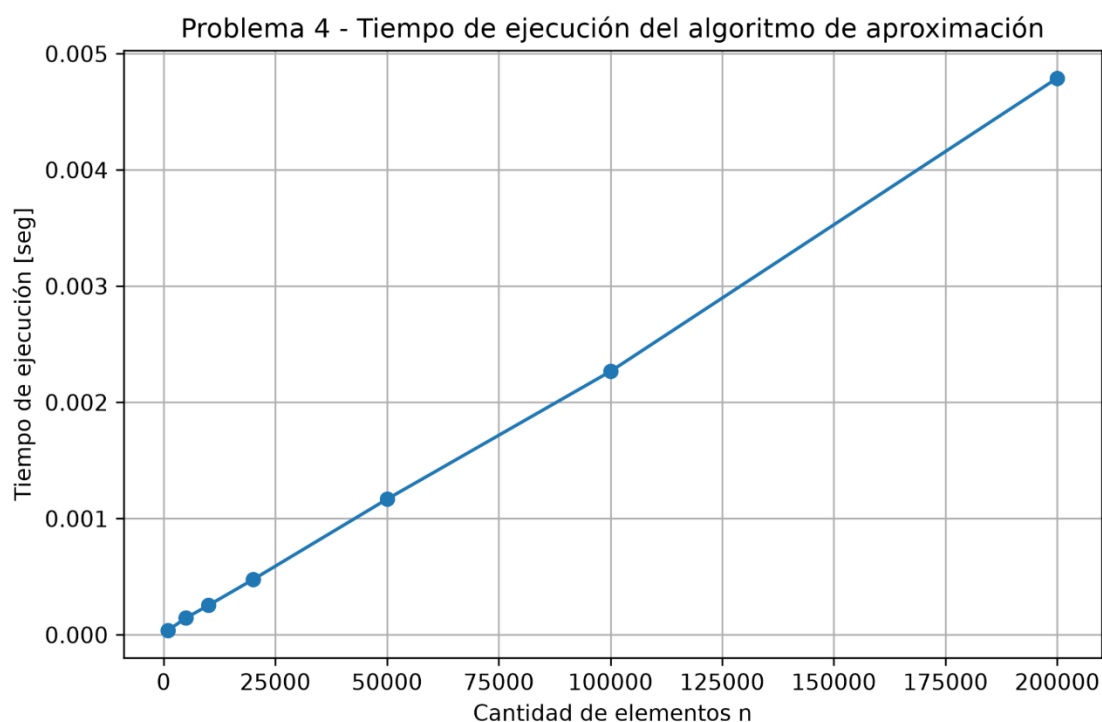
Dada la gran cantidad de datos involucrados en las pruebas de estrés, los *datasets* completos y sus resultados detallados no se transcriben en este documento, sino que se adjuntan en la carpeta de entrega *resultados_subset_sum/*. Allí se pueden consultar individualmente los archivos de entrada generados (por ejemplo, *dataset_1000.txt*) y sus respectivas respuestas (*solucion_1000.txt*).

8. Tiempos de Ejecución

Es importante destacar que los tiempos reportados a continuación corresponden exclusivamente a la ejecución de la función analítica *aproximacion_subconjunto()*. Las operaciones de entrada/salida (como la escritura de archivos en disco, la generación de reportes y la visualización mediante gráficos) fueron excluidas de la medición para evitar sesgar el análisis experimental del algoritmo.

A continuación, se presenta la tabla resumen con los resultados obtenidos para distintos tamaños de entrada n , evaluados contra una capacidad límite constante de $B = 10.000.000$:

n	Cantidad de elementos en la solución	Suma obtenida	Tiempo de ejecución (s)
1000	1000	5086966	0.00003998
5000	1962	9999999	0.00014529
10000	1992	10000000	0.00025334
20000	2015	10000000	0.00047565
50000	1960	10000000	0.00116769
100000	1974	10000000	0.00226650
200000	2028	10000000	0.00479011



Análisis de los resultados experimentales:

La complejidad temporal teórica deducida en el inciso 6 es $O(n)$, producto de realizar un único recorrido sobre el arreglo de entrada. Al observar los datos recolectados y el gráfico de tiempos experimentales, se evidencia un comportamiento marcadamente lineal en la práctica.

Esta correspondencia se verifica al notar que, al duplicar la cantidad de elementos procesados (por ejemplo, al pasar de 50.000 a 100.000, o de 100.000 a 200.000), el tiempo de ejecución también se incrementa aproximadamente al doble. La curva de la gráfica

presenta una forma continua cercana a una recta, sin mostrar evidencias de curvatura típica de un crecimiento cuadrático o exponencial. Por lo tanto, se concluye de forma categórica que los resultados empíricos son consistentes y respaldan la complejidad temporal teórica $O(n)$ establecida en el diseño.

9. Informe de Resultados

El diseño, desarrollo y análisis empírico del algoritmo de aproximación propuesto para la variante del problema de Suma de Subconjuntos (*Subset Sum*) arroja resultados sumamente positivos y plenamente consistentes con las hipótesis teóricas planteadas.

El enfoque de combinar dos heurísticas evaluadas en paralelo demostró ser una estrategia algorítmica robusta. Al integrar el recorrido secuencial (Greedy) con el rastreo continuo del mayor elemento individual, y finalmente devolver el máximo entre ambas opciones, garantizamos que la solución sea siempre estrictamente factible. Dado que ambas sub-soluciones se construyen filtrando cualquier valor que exceda la capacidad B , el resultado devuelto jamás viola las restricciones de capacidad del modelo.

Logramos mitigar por completo la debilidad estructural del algoritmo original, donde la inclusión temprana de un elemento de poco peso podía bloquear el ingreso de valores mucho más significativos. La demostración matemática por reducción al absurdo (validada empíricamente en nuestros casos de prueba, como el escenario problemático de $A=\{1, 100\}$) confirma que la cota de aproximación se cumple de forma estricta. El algoritmo asegura que, en el peor de los escenarios posibles, la suma obtenida siempre representará al menos el 50% del valor del subconjunto factible óptimo: $ALG \geq \frac{OPT}{2}$

Desde la perspectiva del rendimiento computacional, los ensayos de estrés realizados con conjuntos de datos de hasta 200.000 elementos ratifican la altísima eficiencia de la solución propuesta. Al procesar los elementos sobre la marcha, sin requerir costosos ordenamientos previos (que elevarían la complejidad a $O(n \log n)$) ni grandes matrices de programación dinámica (que harían colapsar la memoria del sistema para capacidades B muy grandes), el algoritmo mantiene una estricta complejidad temporal lineal $O(n)$. La gráfica de tiempos de ejecución corrobora empíricamente este comportamiento analítico, trazando una curva de crecimiento marcadamente lineal donde el tiempo de procesamiento escala de forma puramente proporcional al tamaño de la entrada.

En definitiva, la solución desarrollada logra un equilibrio técnico ideal: ofrece una cota de calidad de resultado matemáticamente garantizada y lo hace con el costo computacional asintótico más bajo posible ($O(n)$), convirtiéndose en una herramienta sumamente eficiente y escalable para instancias de procesamiento de datos a gran escala.

Algoritmos Aleatorios

Introducción

El problema de 3-Coloreo consiste en asignar uno de tres colores posibles a cada vértice de un grafo. En su versión clásica de decisión, se busca determinar si existe una asignación de colores tal que toda arista tenga extremos de colores distintos.

En este trabajo se considera la versión de optimización del problema. Dado un grafo $G=(V,E)$, se desea colorear todos sus vértices utilizando únicamente tres colores, maximizando la cantidad de aristas satisfechas. Se dice que una arista $e=(u,v)$ está satisfecha cuando los vértices u y v reciben colores distintos.

Sea c^* la cantidad máxima de aristas que pueden satisfacerse mediante algún 3-coloreo del grafo. El objetivo es diseñar un algoritmo aleatorio de tiempo polinomial cuyo número esperado de aristas satisfechas sea al menos $2/3$ de c^* .

Diseño del algoritmo

Idea general

La idea del algoritmo es asignar a cada vértice uno de los tres colores posibles de forma aleatoria e independiente.

A primera vista podría parecer una estrategia demasiado simple para obtener una aproximación al óptimo. Sin embargo, mediante un análisis probabilístico puede demostrarse que la cantidad esperada de aristas satisfechas alcanza al menos dos tercios de la solución óptima.

Descripción

1. Para cada vértice del grafo:
 - a. Se selecciona aleatoriamente uno de los tres colores disponibles.
 - b. Se almacena el color asignado.
 2. Una vez coloreados todos los vértices, se recorren las aristas.
 3. Se contabilizan aquellas cuyos extremos poseen colores diferentes.
- El resultado final es el coloreo de vértices y la cantidad de aristas satisfechas obtenida.

Pseudocódigo

```
ALGORITMO TRES_COLOREO_ALEATORIO(G)
Entrada:
G=(V,E)
Salida:
coloreo aleatorio
cantidad de aristas satisfechas

    color ← ASIGNAR_COLORES(V)
    satisfechas ← CONTAR_SATISFECHAS(E, color)
    devolver (color, satisfechas)

SUBALGORITMO ASIGNAR_COLORES(V)
    crear diccionario color
    para cada vértice v ∈ V:
        color[v] ← número aleatorio entre {1,2,3}
    devolver color

SUBALGORITMO CONTAR_SATISFECHAS(E, color)
    satisfechas ← 0
    para cada arista (u,v) ∈ E:
        si color[u] ≠ color[v]:
            satisfechas ← satisfechas + 1
    devolver satisfechas
```

Estructuras de datos utilizadas

Las estructuras empleadas son:

- Lista de vértices.
- Lista de aristas.
- Diccionario o arreglo para almacenar el color de cada vértice.
- Contador de aristas satisfechas.

La elección de estas estructuras permite recorrer tanto los vértices como las aristas en tiempo lineal.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

- Python Software Foundation. (2021). Python 3.10.0 documentation. <https://docs.python.org/3.10/>
- Rivas, A. (2022). Cómo hacer una lista de referencias con Normas APA. Guía Normas APA. <https://normasapa.in/como-hacer-la-lista-de-referencias/>
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- Graham, R. L., Knuth, D. E., & Patashnik, O. (1994). *Concrete mathematics: A foundation for computer science* (2nd ed.). Addison-Wesley Professional.
- Knuth, D. E. (1997). *The art of computer programming, Vol. 2: Seminumerical algorithms* (3rd ed.). Addison-Wesley Professional.
- Levitin, A. (2011). Introduction to the design and analysis of algorithms (3rd ed.). Pearson.
- Kleinberg, J., & Tardos, É. (2005). Algorithm design. Pearson Addison-Wesley.
- Kumar, V. (1992). Algorithms for constraint satisfaction problems: A survey. AI Magazine, 13(1), 32–44. <https://doi.org/10.1609/aimag.v13i1.994>
- Nilsson, N. J. (1998). Artificial intelligence: A new synthesis. Morgan Kaufmann.
- Bellman, R. (1957). Dynamic programming. Princeton University Press.
- Manacher, G. (1975). A new linear-time "on-line" algorithm for finding the smallest initial palindrome of a string. Journal of the ACM, 22(3), 346–351. <https://doi.org/10.1145/321892.321896>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Van Rossum, G., & Drake, F. L. (2009). Python 3 reference manual. CreateSpace.